Python Programming Lab. [Batch-B3]

Assignment Submission Details

Field	Details
Student Name *	Dilip Balu Magar
Student PRN *	125M1H055
Course Name	Python Programming Lab. (MCA31PC06)
Academic Year	2025-26 (Semester-1)
Course Teacher	Prof. Prakash Ukhalkar
Assignment Name	Assignments based on Input Output and File Handling using Python 2.
Assignment Number	Assignment 11
Submission Date *	30-11-2025

1.Write a Python program that reads a CSV file containing student names and their corresponding scores. Calculate the average score and print the names of students with scores above the average.

In [4]:
```
%%writefile students.csv
Name,Score
Dilip,78
Harsh,90
Harry,65
Tushar,88
Sudhir,75
Harshal,50
Gopi,60
Gopa,70
```

```
Overwriting students.csv
```

In [5]:
```
import csv

filename = "students.csv"

try:
    scores = []
    students = []

    with open(filename, "r") as file:
        reader = csv.reader(file)
```

```
        header = next(reader)       # read header
        print("File Data:\n")
        print(header)               # print header row
        for row in reader:
            print(row)              # print each row (raw data)

            name = row[0]
            score = float(row[1])

            students.append((name, score))
            scores.append(score)

    # Calculate average
    avg = sum(scores) / len(scores)
    print("\nAverage score:", avg)

    # Students above avg
    print("\nStudents with score above average:")
    for name, score in students:
        if score > avg:
            print(name, "-", score)

except FileNotFoundError:
    print("Error: File", filename, "not found.")
```

```
File Data:

['Name', 'Score']
['Dilip', '78']
['Harsh', '90']
['Harry', '65']
['Tushar', '88']
['Sudhir', '75']
['Harshal', '50']
['Gopi', '60']
['Gopa', '70']

Average score: 72.0

Students with score above average:
Dilip - 78.0
Harsh - 90.0
Tushar - 88.0
Sudhir - 75.0
```

2.Write a Python program that appends a timestamp to each line of an existing text file, indicating when each line was last modified.

In [ ]:
```
from datetime import datetime

filename = "notes.txt"

try:

    with open(filename, "r") as file:
        lines = file.readlines()  # list of lines
```

```
    print("Original file content:\n")
    for line in lines:
        print(line, end="")
        # end="" prevents print() from adding an extra newline

    # adding timestamp to each line
    updated_lines = []
    for line in lines:
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        updated_lines.append(f"{timestamp} {line}")


    with open(filename, "w") as file:
        file.writelines(updated_lines)


    print("\nUpdated file content :\n")
    with open(filename, "r") as file:
        print(file.read())

except FileNotFoundError:
    print("Error:", filename, "not found.")
```

```
Original file content:

this is the first line
this is the second line
this is the third line
this is the last line of notes.txt


Updated file content :

2025-11-29 23:43:40 this is the first line
2025-11-29 23:43:40 this is the second line
2025-11-29 23:43:40 this is the third line
2025-11-29 23:43:40 this is the last line of notes.txt
```

3.Create a program that reads a binary file and counts the occurrences of a specific byte pattern. Handle the IOError exception and print a message if there is an issue with file I/O.

In [ ]:
```
pattern = b"%PDF"
# as all pdf file contains this at start of file

filename = "File_Handling.pdf"

try:
    with open(filename, "rb") as file:
        content = file.read()

    count = content.count(pattern)
    print("Pattern", pattern, "occurs", count, "time")
```

```
except IOError:
    print("I/O error: Could not read file", filename)
```

```
Pattern b'%PDF' occurs 1 time
```

4.Write a Python program that reads a JSON file containing a list of books. Allow the user to search for a book by title and print its details if found.

In [11]:
```
import json

books = [
    {"title": "Python", "developer": "Guido van Rossum", "year": 1991},
    {"title": "Java", "developer": "James Gosling", "year": 1995},
    {"title": "JavaScript", "developer": "Brendan Eich", "year": 1995},
    {"title": "C", "developer": "Dennis Ritchie", "year": 1972},
    {"title": "C++", "developer": "Bjarne Stroustrup", "year": 1985}
]


with open("books.json", "w") as file:
    json.dump(books, file, indent=4)

print("books.json created")
```

```
books.json created
```

In [ ]:
```
import json

filename = "books.json"

try:
    with open(filename, "r") as file:
        books = json.load(file)  #converts JSON into a Python list of dictionaries

    title = input("Enter book title to search: ").lower()

    found = False
    for book in books:
        if book.get("title", "").lower() == title:
            #If "title" does NOT exist returns "" , not an error
            print("\nBook found:")
            for key, value in book.items():
                print(f"{key}: {value}")
            found = True
            break

    if not found:
        print(f"{title} Book not found.")

except FileNotFoundError:
    print("Error: books.json file not found.")
except json.JSONDecodeError:
    print("Error: Invalid JSON format in file.")
```

```
javac Book not found.
```

Book found: title: Java developer: James Gosling year: 1995

5.Write a Python program that creates a new directory and then creates multiple text files inside that directory, each with a different name and content..

In [15]:
```python
import os

dir_name = "my_files"

# Create directory if it doesn't exist
os.makedirs(dir_name, exist_ok=True)

files_data = {
    "file.txt": "This is the content of file.",
    "file1.txt": "This is the content of file 1.",
    "file2.txt": "This is the content of file 2.",
}

# Create files and display their content
for fname, content in files_data.items():
    filepath = os.path.join(dir_name, fname)

    # Create and write content
    with open(filepath, "w") as file:
        file.write(content)

    # Display file name and content
    print(f"\nCreated file: {fname}")
    print("Content:")
    with open(filepath, "r") as file:
        print(file.read())

print("\nAll files created in directory:", dir_name)
```

```
Created file: file.txt
Content:
This is the content of file.

Created file: file1.txt
Content:
This is the content of file 1.

Created file: file2.txt
Content:
This is the content of file 2.

All files created in directory: my_files
```

6.Create a program that reads a log file and extracts information about errors. Print the count of each type of error along with the corresponding error messages.

In [18]:
```
%%writefile log.txt
INFO: System started
```

```
ERROR: File not found
ERROR: Network failure
WARNING: Low memory
ERROR: File not found
ERROR Something went wrong
```

```
Overwriting log.txt
```

In [19]:
```
filename = "log.txt"
error_counts = {} ## Dictionary to store error messages and how many times they app


try:
    with open(filename, "r") as file:
        for line in file:
            line = line.strip() ## Remove extra spaces and newline characters
            if line.startswith("ERROR"):
               #  # Split the line into two parts at the first colon only
                # Example: "ERROR: File not found"  then ["ERROR", " File not found
                parts = line.split(":", 1)
                if len(parts) > 1:
                    message = parts[1].strip()
                else:
                    message = "Unknown error"
                #urrences of each error message
                error_counts[message] = error_counts.get(message, 0) + 1
                  # .get(message, 0) returns 0 if message does not exist yet


    if error_counts:
        print("Error summary:\n")
        for msg, count in error_counts.items():
            print(f"{msg} -> {count} time")
    else:
        print("No errors found in log file.")

except FileNotFoundError:
    print("Error: log file not found.")
```

```
Error summary:

File not found -> 2 time
Network failure -> 1 time
Unknown error -> 1 time
```

7.Create a program that reads a text file containing a list of email addresses. Identify and print all unique domains present in the email addresses.

In [20]:
```
%%writefile emails.txt
dilip@gmail.com
magar@yahoo.com
harsh@outlook.com
harry@gmail.com
sudhir@college.edu
student@pccoepune.org
```

```
Writing emails.txt
```

In [21]:
```python
filename = "emails.txt"
domains = set()     # Using a set because sets store only unique values

try:
    # Open the file for reading only
    with open(filename, "r") as file:
        for line in file:

            email = line.strip()   # Remove spaces and newline at end

            # Only process valid emails that contain '@'
            if "@" in email:
                # Split email into username and domain
                # Example: "dilip@gmail.com" then ["dilip", "gmail.com"]
                _, domain = email.split("@", 1)

                domains.add(domain)   # Add domain to set duplicates ignored

    print("Unique domains found:\n")
    for d in domains:
        print(d)

except FileNotFoundError:
    print("Error: emails.txt not found.")
```

```
Unique domains found:

pccoepune.org
college.edu
yahoo.com
gmail.com
outlook.com
```

8.Write a Python program that reads a binary file and performs encryption on its contents. Save the encrypted data to a new file.

In [22]:
```python
input_file = "File_Handling.pdf"
output_file = "File_Handling2.pdf"
key = 0x55                            # XOR key

try:

    with open(input_file, "rb") as infile, open(output_file, "wb") as outfile:

        data = infile.read()

        # Encrypt using XOR:
        # XOR flips bits using the key.
        # b ^ key means: take each byte from data and XOR it with key.
        encrypted = bytes([b ^ key for b in data])
        # We use bytes([...]) because XOR returns a list of integers,
        # and bytes() converts the list back into a byte sequence.

        outfile.write(encrypted)
```

```
    print("Encryption done. Encrypted file:", output_file)


except FileNotFoundError:
    print("Error:", input_file, "not found.")


except IOError:
    print("I/O error during encryption.")
```

```
Encryption done. Encrypted file: File_Handling2.pdf
```

9.Write a Python program that accepts user input for a file path and checks whether the file exists. If it does, print the file's size; otherwise, print a message indicating that the file doesn't exist.

In [25]:
```
import os

path = input("Enter file path: ")
print("file path:",path)

if os.path.exists(path):
    size = os.path.getsize(path)
    print("File exists.")
    print("File size:", size, "bytes")
else:
    print("File does not exist.")
```

```
file path: file path: 125M1H055_A11_PP_LAB_B3.ipyn
File does not exist.
```

file path: 125M1H055_A11_PP_LAB_B3.ipynb File exists. File size: 20700 bytes

10. Create a program that reads a JSON file containing information about employees. Allow the user to add a new employee to the file and save the updated data.

In [27]:
```
import json

employees = [
    {"id": 1, "name": "Amit", "department": "IT"},
    {"id": 2, "name": "Sudhit", "department": "HR"},
    {"id": 3, "name": "Dilip", "department": "Finance"},
]

filename = "employees.json"

with open(filename, "w") as file:
    json.dump(employees, file, indent=4)
    # indent=4 makes the JSON file well formatted and readable

print("JSON file created successfully:", filename)


print("\nContents of employees.json:\n")
```

```
with open(filename, "r") as file:
    data = json.load(file)     # Convert JSON file to  Python list
    print(json.dumps(data, indent=4))
```

```
JSON file created successfully: employees.json

Contents of employees.json:

[
    {
        "id": 1,
        "name": "Amit",
        "department": "IT"
    },
    {
        "id": 2,
        "name": "Sudhit",
        "department": "HR"
    },
    {
        "id": 3,
        "name": "Dilip",
        "department": "Finance"
    }
]
```

In [29]:

```
import json
import os

filename = "employees.json"


if os.path.exists(filename):
    try:
        with open(filename, "r") as file:
            employees = json.load(file)
            # json.load converts JSON data into a Python list
    except json.JSONDecodeError:

        employees = []
else:
    employees = []


print("Enter new employee details:")
emp_id = int(input("ID: "))
name = input("Name: ")
dept = input("Department: ")

# Create a dictionary for the new employee
new_employee = {
    "id": emp_id,
    "name": name,
    "department": dept
}
```

```python
# Add this dictionary to the list of employees
print("new employee details",new_employee)
employees.append(new_employee)

with open(filename, "w") as file:
    json.dump(employees, file, indent=4)
    # json.dump  writes list to file in JSON format

print("New employee added and data saved to", filename)

print("\nUpdated employees.json content:\n")

with open(filename, "r") as file:
    updated_data = json.load(file)
    print(json.dumps(updated_data, indent=4))
```

```
Enter new employee details:
new employee details {'id': 55, 'name': 'Harsh', 'department': 'CS'}
New employee added and data saved to employees.json

Updated employees.json content:

[
    {
        "id": 1,
        "name": "Amit",
        "department": "IT"
    },
    {
        "id": 2,
        "name": "Sudhit",
        "department": "HR"
    },
    {
        "id": 3,
        "name": "Dilip",
        "department": "Finance"
    },
    {
        "id": 55,
        "name": "Harsh",
        "department": "CS"
    }
]
```